A barrier-function inventory guard for a learned market maker

Brandon-Jason Adebayo · November 2026 · code and data recorder: github.com/B4ryterx/safe-market-maker

Abstract

I trained a linear market-making policy by trial and error (the cross-entropy method) on one day of Bitfinex BTCUSD order-by-order data and tested it, with queue-aware fills, on three later recordings that played no part in training or tuning (49 complete hours). Two questions: does the learned policy beat the textbook Avellaneda–Stoikov formula, and what does a control-barrier-function (CBF) guard on inventory cost? The guard result is strong: every guarded policy stayed inside its inventory limit in every test, while the plain hard cap breached it under order latency and unguarded policies reached 3 to 11 times the limit. I also found that the guard's guarantee, as first written, only holds without latency; counting quotes still in flight restores it at no measurable cost. The profit result is weak: the learned policy beats the tuned textbook bot by $2.56 an hour (t = 5.2) but is level with a bot that simply joins the best bid and ask (t = 0.2), and all of them lose money over the 49 hours.

1. Setup

Data. A recorder (`tools/record_l3.py`) stores Bitfinex's raw book (every order by id), every trade and the exchange's checksums. Recording 1 (12 h) is used for training and validation; recordings 2, 3 and 4 (12 h, 21.6 h and 21 h, made on different days) are the test set. Rebuilt books match every exchange checksum (29,397 on recording 3 alone). The market is thin: roughly one trade every three to four seconds, median trade under $20.

Fills. Our quotes are virtual and do not move the market. A quote joins the back of the queue at its price and fills only when a trade prints through its price, or a trade at its price hits an order that joined the queue after ours. Orders resting ahead of us must trade or cancel first. Quotes are passive only. Profit on each fill is split exactly into *spread* (distance from the mid when filled), *adverse selection* (mid move over the next 10 s) and *inventory* (mid move after that).

Bots. Order size 0.001 BTC, inventory limit Q = 0.01 BTC (10 orders), one decision per second, zero fees. (1) *Simple*: join the best bid and ask. (2) *Textbook*: Avellaneda–Stoikov with rolling volatility and an imbalance term, three parameters grid-tuned on recording 1. Both use a hard cap (stop quoting the side that would breach the limit). (3) *Learned*: a linear map from six features (inventory, book imbalance, 10 s price move, spread, volatility, constant) to a bid and an ask offset in ticks, trained by the cross-entropy method (96 policies per generation, 25 generations) to maximise profit minus a quadratic inventory penalty on the first 8 hours of recording 1; the last 4 hours choose the generation.

2. The barrier guard

Safe set $h(q) = Q^2 - q^2 \geq 0$ on inventory q. The discrete-time barrier condition $h(q^+) \geq (1 - \alpha)\, h(q)$ is enforced against the worst case that one side's entire posted size fills before the next decision. Solving for the size gives

$$\text{bid size} \leq \sqrt{\alpha Q^2 + (1-\alpha)q^2} - q, \qquad \text{ask size} \leq \sqrt{\alpha Q^2 + (1-\alpha)q^2} + q.$$

The guard leaves the policy alone while these hold (far from the limit the bound exceeds one order), shrinks only the risky side near the limit, and $\alpha = 1$ is the hard cap.

Guarantee without latency. If $|q| \leq Q$ at a decision then $b = \sqrt{\alpha Q^2 + (1-\alpha)q^2} \leq Q$, and any sequence of fills before the next decision leaves q at most q + bid size $\leq b \leq Q$ (and symmetrically at least $-Q$). By induction $|q| \leq Q$ forever.

The gap: latency. With latency L, a replaced quote stays live until its cancel lands, and the new quote is already committed. Both were sized against inventory that ignored the other, so both can fill. A unit test (`tests/test_sim.py`) builds an adversarial tape where every posted bid fills: with 1 s latency the plain guard breaches the limit for $\alpha \geq 0.4$ (reaching 1.1Q at $\alpha = 1$). At $\alpha = 0.2$, the published setting, it does not, because each quote takes only a fraction of the room left: two quotes sized at the same q satisfy $q + 2s(q) \leq Q$ whenever

$q \geq (4\alpha - 1)Q/(3 - 4\alpha)$, and below that point the one-order size cap binds. That explains why the published latency runs never breached, but it is a property of small α, not a guarantee.

Fix. Size each side from the worst case after everything already in flight fills: replace q by q + I_bid in the bid bound (q − I_ask in the ask bound), where I is the remaining size of quotes not yet live or not yet cancelled. If $q + I \leq Q$ at one decision, then at the next the new in-flight size is at most the old in-flight size plus the quote just sized, so $q^+ + I^+ \leq q + I +$ bid size $\leq Q$. The guarantee holds for any α, provided latency is at most one decision interval. The adversarial test confirms it for α from 0.05 to 1.

3. Results

All held-out data (recordings 2–4, 49 complete hours; `stress_base*.csv`). ± is the standard error over hourly blocks, which treats hours as independent and is therefore generous.

Bot	Total	Per hour	vs simple (t)	vs textbook (t)	Mean inventory (orders)	Seconds over limit
Simple	–$42.36	–$0.87 ± 0.36	—	+$2.47 (4.1)	7.6	0
Textbook (tuned A–S)	–$163.43	–$3.34 ± 0.79	–$2.47 (–4.1)	—	7.6	0
Learned, no guard	–$103.46	–$2.11 ± 1.19	–$1.25 (–1.1)	+$1.22 (1.3)	6.2	25,723
Learned, guard in training	–$37.99	–$0.78 ± 0.37	+$0.09 (0.2)	+$2.56 (5.2)	3.0	0

The pattern held on each recording separately. On recording 2 (the first test) the guarded policy made +$5.70 against +$3.49 for the simple bot; that lead did not repeat on recordings 3 and 4, where every bot lost money (the price rose 1.2% during recording 3 and the bots were short into it).

What the guard buys. Without it, the learned policy went to 3.8 times the limit on recording 3 and 7.4 times on recording 4; five retrained unguarded seeds went 3 to 11 times over. The same no-guard policies with the guard added only at test time stayed inside and lost far less (–$43.78 vs –$103.46 for the published one). In training the guard mattered even more: the median first-generation policy scored about –$714 unguarded and –$2.50 guarded.

Latency and the queue model (published guard-trained policy; recordings 2–3 / recording 4).

Scenario	Learned + guard	Simple	Simple's worst position
No latency	+$2.46 / –$39.64	–$4.93 / –$35.84	at the limit
1 s to place or cancel, plain guard	–$1.34 / –$76.26	–$6.87 / –$63.38	1.09Q / 1.10Q, over for 641 s / 1,595 s
1 s, guard counts in-flight quotes	–$1.08 / –$73.84	(unchanged)	
Fill only on trades through our price	–$1.21 / –$50.31	–$11.58 / –$46.86	at the limit

Under the pessimistic queue the learned policy fell behind the simple bot on recording 4. The hard cap breaches under latency because it only stops quoting at the limit while an uncancelled quote can still fill. The in-flight barrier was as good as or better than the plain one in 22 of 24 policy-recording pairs (mean +$0.45 and +$1.06), at about 3% lower inventory: the guarantee is free.

Robustness checks. Five retrained seeds per variant gave the same ordering (every learned policy beat the textbook bot, t 1.7 to 2.9 on recordings 2–3; none beat the simple bot clearly). The weak step was choosing the generation: two of five seeds kept the barely trained generation 0. Averaging the last ten generations' weights (no validation data needed) cut the spread of outcomes across seeds from $9.59 to $1.00 on recordings 2–3 and from $14.85 to $2.01 on recording 4. A volatility-aware barrier, which shrinks Q when one-second mid changes are larger than on the training data, beat the fixed barrier for all six fixed-barrier policies on every test, mainly through smaller worst hours.

4. What this shows, and what it does not

- **Shown:** a barrier on inventory gives a hard limit with minimal intervention, survives latency once in-flight quotes are counted, and makes trial-and-error training far more stable. On held-out data the learned policy

carries about 60% less inventory than either baseline (3.0 orders against 7.6) and decisively beats the tuned textbook formula.

- **Not shown:** that the learned policy makes money. Over 49 hours it is level with the simplest possible bot, and at Bitfinex's standard maker fee (10 bps) or even 1 bps every bot loses heavily: the gross edge is a fraction of a basis point of volume.
- **Limits.** One thin market and about 55 test hours; no market impact (the policy often improves the best price by one tick, and real competitors would react); queue position within a price level is inferred from the feed, not confirmed by the exchange; the generation-choice rules and the volatility reference were chosen after seeing earlier results, so they carry a mild selection effect; hourly standard errors ignore autocorrelation. The test comparison on recording 2 was once run by mistake while training was still in progress; nothing was changed afterwards.

Reproduce

`make test` runs 27 unit tests on synthetic books in under a second (queue position, trades through our price, the pessimistic queue, passive-only quotes, the profit split, the guard's invariant, and the latency gap and its fix). `make check` reproduces the published recording-2 numbers; `make results` regenerates every table from the recordings and saved policies; `make retrain` retrains the policies.